

GODOT
Game engine

Il Manuale

Corso di Godot

Corso a cura del prof. Nicola Regge

Perché quello che impari qui può cambiarti le cose

Due parole mie, prima di cominciare.

Voglio dirti una cosa, dritta, prima di tutto il resto. Alla tua età **nessuno si aspetta che tu sia già un esperto**. Chi ti darà il primo tirocinio, il primo lavoro, non guarderà per prima cosa quanto codice sai scrivere: quello te lo insegnano. Sceglierà **te**, e non un altro, soprattutto per **due cose**.

La prima: come ti poni. Se ci sei ogni mattina, in orario. Se **rispondi** quando qualcuno ti scrive. Se, quando sbagli, hai il coraggio di dire “ho sbagliato, lo aggiusto” invece di nasconderti. Se **non molli** al primo “no”. Se sai stare in squadra. Non sono cose “da grandi”: sono cose che **puoi decidere di fare da domani**, e per me pesano più di mille nozioni.

La seconda: cosa sai fare e mostrare. Non parole – roba fatta **con le tue mani**. Un gioco che gira. Una cosa che apri sul telefono e dici, guardando negli occhi chi hai davanti: **“Questo l’ho fatto io.”**

Queste due cose **insieme** fanno scegliere te. Per questo non ti faccio fare esercizi da scuola e basta: ogni cosa che costruisci, anche piccola, va nel **tuo** quaderno. Pezzo dopo pezzo ti costruisci una prova – che **sai fare**, e che su di te si può contare.

E adesso la cosa che mi sta più a cuore. Quest’anno lo passiamo **insieme**: ci scherzeremo, giocheremo, cresceremo. Certi giorni ci abbracceremo, altri litigheremo – ci mostreremo pure i denti. Ti spingerò forte, ti farò arrabbiare, ti darò voti che non ti piacciono, e sì, potrei perfino doverti bocciare. Ma **qualunque cosa succeda, sono dalla tua parte**: se sono duro non è contro di te, è perché **credo in te** e voglio il meglio che puoi diventare.

C’è un patto, però. **In laboratorio siamo professionisti**, con rispetto dei ruoli: io il tuo **formatore**, tu il mio **allievo**. Fuori, al bar, è un’altra musica: ci scappa la battuta, ci prendiamo pure in giro – ma sempre con rispetto. E **davanti agli altri** siamo sempre educati: chi ci vede non sa che rapporto abbiamo, e la prima impressione parla di **tutti e due**.

Molti di voi partono da lontano, da porte che si sono chiuse. Questa è una porta che **si apre**, e te la apro io. Non sprecarla: prenditi ogni piccola vittoria. Il resto – il lavoro, il rispetto, le occasioni – nasce da lì. Ci credo, e ci credo **in te**.

Nicola

Come si valutano i compiti

Le regole del gioco, chiare fin da subito.

Qui non ti freghiamo: sai **in anticipo** come funziona la valutazione. Leggila una volta, poi lavora tranquillo.

1. Conta prima di tutto come ti poni. L'atteggiamento pesa quanto e più della tecnica: esserci in orario, non mollare al primo errore, dire "ho sbagliato" invece di nasconderti, dare una mano ai compagni. È la prima cosa che guardo — ed è la stessa che guarderà, un domani, chi ti darà un lavoro.

2. Il codice che funziona è il biglietto d'ingresso, non il voto. Consegnare il gioco che gira ti fa "entrare alla prova". Ma il codice **da solo** non fa il voto: puoi copiarlo o fartelo scrivere... e allora non direbbe niente su di te.

3. Il voto nasce dalla prova dal vivo, da solo. Uno di questi due, o tutti e due:

- **Me lo spieghi:** racconti **a parole tue** cosa fa il tuo codice.
- **Il gioco rotto:** ti do il tuo gioco con **2-3 errori nascosti** dentro, e tu lo **rimetti in piedi lì per lì** — da solo, senza AI e senza chiedere ai compagni.

Se hai capito, li fai in pochi minuti. Se hai solo incollato, ti blocchi. Ecco perché **capire conviene**.

4. Il patto con l'AI e con i compagni. Puoi usarli per **imparare**: capire un errore, farti spiegare, avere uno spunto. Non per **consegnare senza capire**. La prova del nove è sempre la stessa: **lo sai spiegare e riparare da solo?**

5. Zero vergogna. Usare gli aiuti, come i 4 livelli, l'AI o un compagno, è **permesso e normale** — non è imbrogliare. Anche sbagliare è normale: **capita a tutti i programmatori**, pure ai più bravi. L'unica cosa che conta è che, alla fine, **tu abbia capito**.

Scrivere in Markdown

La tua dispensa, fatta con due segnetti.

Markdown, si dice “*marc-daun*”, è un modo per scrivere **testo normale** e aggiungere qualche **segnetto** che dice *come deve apparire*: questo è un titolo, questa parola è in grassetto, questo è un elenco.

***Il ponte con quello che conosci:** in Word premi il bottone grassetto; qui il bottone non c'è, il grassetto lo **scrivi tu** mettendo due asterischi attorno alla parola. Tutto qui. I segnetti li scrivi tu, ma nel risultato finale **non si vedono**.*

La tabella dei segnetti

Vuoi ottenere...	Scrivi così
Un titolo grande	# Il mio titolo
Un titolo più piccolo	## Sottotitolo — più #, più piccolo
Grassetto	**parola** — due asterischi prima e dopo
<i>Corsivo</i>	*parola* — un asterisco prima e dopo
Un punto elenco	- prima cosa — trattino più spazio
Un elenco numerato	1. prima cosa
Un nome tecnico / codice in riga	`_process` — un apice basso prima e dopo
Un'immagine, un tuo screenshot	![il mio gioco](immagini/es1.png)
Una nota/riquadro	> Attenzione: ricordati di salvare!

L'apice basso ```, in inglese *backtick*, su tastiera italiana si fa con **Alt Gr** + `'`, il tasto dell'apostrofo vicino allo `0`.

Un blocco di codice

Metti **tre apici bassi** prima e **tre dopo**: così il codice viene mostrato bello incolonnato.

```
```gdscript
func _ready():
 print("Ciao!")
```
```

Le 4 regole d'oro

1. **Riga vuota tra un paragrafo e l'altro.** Se non la metti, le frasi si attaccano tutte insieme.
2. **Uno spazio dopo # e dopo -.** `#Titolo` non funziona, `# Titolo` sì.
3. **I segnetti non si vedranno:** servono solo a dare la forma. Non spaventarti se nel testo grezzo sembrano strani.
4. **Bloccato? Fatti aiutare bene dall'AI.** Scrivi con **parole tue**, poi chiedi “*mettimi questo in Markdown*”, e **guarda come l'ha fatto** — così impari il segnetto per la prossima volta. Ricorda il patto: l'AI ti aiuta a *formattare*, il pensiero resta tuo.

Come parti da un modello

Non parti mai da un foglio bianco: c'è un **modello** già pronto, in inglese *template*, con i titoli e i posti da riempire.

Cos'è il “repository”? È solo una parola tecnica per dire **la cartella del corso su GitHub**, dove sono raccolti tutti i file: il manuale, gli esercizi, i modelli. È lo stesso posto che gestiamo con **Git**. Da browser lo apri come un sito qualsiasi: cartelle e file su cui puoi cliccare.

Ecco come apri il modello e te ne fai **una copia tua**:

1. `[BROWSER]` apri la pagina del corso su GitHub. L'indirizzo esatto te lo do io.
2. Nella lista, clicca prima la cartella `manuale`, poi il file `quaderno-studente-TEMPLATE.md`: si apre e vedi il testo del modello.
3. In alto a destra, sopra il testo, clicca l'iconcina `Copy raw file`, che copia tutto il contenuto in un colpo solo.
4. Torna nel **TUO** repository, la tua copia personale → bottone `Add file` → `Create new file`.
5. Dai un nome che finisce con `.md`, per esempio `es1.md`.
6. **Incolla** con `Ctrl + V` il modello, poi **riempi i vuoti** con le tue cose.
7. Bottone verde `Commit changes` per salvare. Fatto!

Come metto un mio screenshot

Uno screenshot è una **foto dello schermo**. Attenzione: appena lo fai, finisce solo nella memoria temporanea, i cosiddetti “appunti” — **non è ancora un file** sul computer. Ecco tutti i passaggi:

1. [Windows] premi **Win + Shift + S**: lo schermo si scurisce, **trascina** un riquadro attorno al tuo gioco. L'immagine viene copiata.
2. In basso a destra compare un **avviso: cliccaci sopra** → si apre lo **Strumento di cattura**.
3. Lì clicca l'iconcina del **dischetto** per salvare, scegli una cartella che **ricordi bene**, per esempio il **Desktop**, e un nome che finisce con **.png**, per esempio **es1.png**. Ora è un **file** sul tuo computer.
4. [BROWSER] nel tuo repository apri la cartella **immagini/** → bottone **Add file** → **Upload files** → **trascina** dentro il tuo **es1.png**.
5. La riga nel modello è **già pronta**: **![il mio gioco](immagini/es1.png)** → l'immagine comparirà da sola.

Prova del nove: se guardi la tua pagina e vedi il titolo grande, il grassetto e il tuo screenshot al posto giusto... **ce l'hai fatta!**

Cos'è Godot, il parente di Lazarus

Godot è un programma gratuito per creare **giochi** e app interattive. È molto simile, come spirito, a **Lazarus**: entrambi sono ambienti gratuiti dove **componi qualcosa a schermo e ci attacchi del codice**.

La “stele di Rosetta” Lazarus → Godot:

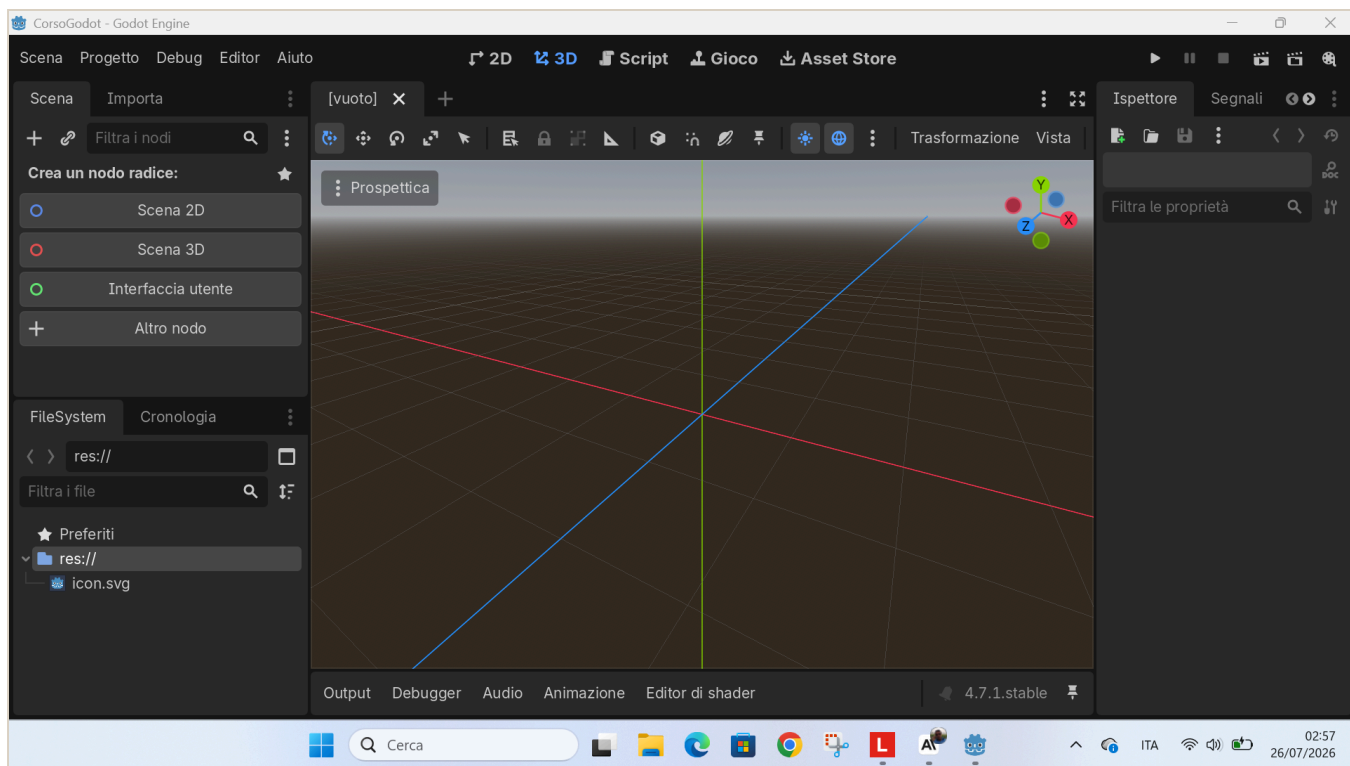
| In Lazarus, che già conosci | In Godot, la novità |
|--|---|
| Progetto | Progetto , identico |
| Form , la finestra | Scena , un <i>albero di nodi</i> più potente |
| Componenti : TButton, TEdit... | Nodi : Button, LineEdit, Label... |
| Proprietà come Caption, nell'Object Inspector | Proprietà nell' Ispettore |
| Gestore evento , <code>Button1Click</code> | Segnale + funzione |
| Object Pascal , il linguaggio | GDScript , in stile Python |

La differenza più importante — il “game loop”: in Lazarus il programma è **fermo** finché non clicchi qualcosa. In Godot c'è una funzione, `_process(delta)`, che **gira da sola ~60 volte al secondo**, in continuazione. È questo che fa muovere le cose: personaggi, oggetti che cadono, animazioni.

*In una frase: **Lazarus reagisce, Godot pulsa.***

L'ambiente di sviluppo all'avvio

Quando apri un progetto nuovo, l'editor si presenta così: vista **3D** di default e scena ancora vuota.



L'editor di Godot appena aperto, con la scena vuota

*Per il nostro corso lavoreremo quasi sempre in **2D**: cliccheremo “**Scena 2D**” per iniziare. Lo vedrai nei capitoli in cui costruiamo gli esercizi.*

I 4 concetti base di Godot

Se capisci questi quattro, capisci Godot:

| Concetto | Cos'è | Analogia LEGO |
|--|---|--------------------|
| Progetto | La cartella con dentro il file <code>project.godot</code> e tutto il gioco | La scatola del set |
| Nodo | Il mattoncino base: Sprite2D per un'immagine, Label per un testo, Timer per un cronometro | Un pezzo di LEGO |
| Scena | Tanti nodi messi ad albero, salvati in un file <code>.tscn</code> | Una costruzione |
| Script , il file <code>.gd</code> | Codice GDScript attaccato a un nodo, che gli dà comportamento | Le istruzioni |

Regola d'oro: in Godot **tutto è un nodo**; le scene sono nodi messi insieme; gli script danno vita ai nodi.

GDScript: il linguaggio

GDScript è la lingua che si parla **dentro** Godot. È stato fatto apposta per somigliare a **Python**: si legge facile, si usa il **rientro con TAB** per raggruppare le righe, **niente punto e virgola**.

Due funzioni speciali che Godot chiama da solo:

- `func _ready():` → eseguita **una volta**, all'avvio, per preparare le cose.
- `func _process(delta):` → eseguita **a ogni fotogramma**: è il game loop. `delta` = secondi passati dall'ultimo fotogramma; serve a muoversi in modo fluido su qualsiasi PC.

Esempio minimo, leggere le frecce e spostare qualcosa:

```
func _process(delta):
    if Input.is_action_pressed("ui_left"):
        posizione.x -= 200 * delta    # vai a sinistra
    if Input.is_action_pressed("ui_right"):
        posizione.x += 200 * delta    # vai a destra
```

Costruiamo l'Esercizio 1: il bottone che saluta

Il tuo primo gioco vero, in pochi passi. Alla fine avrai **un bottone** che, quando lo premi, **ti saluta**. È il ponte da Lazarus: il vecchio `Button1Click` qui diventa un **segnale**.

Passo 1 — Crea il progetto

[APP – Godot] nella finestra iniziale, il Gestore progetti, in alto a destra clicca **Crea**. Dai un nome, per esempio `esercizio1`, scegli una cartella e premi **Crea e modifica**.

Passo 2 — Fai la scena

[APP – Godot] pannello **Scena**, in alto a sinistra, clicca **Scena 2D**: nasce il nodo radice `Node2D`. Con un **doppio clic** sul suo nome, rinominalo **Main**.

SCREENSHOT DA INSERIRE

La scena con il nodo Main
immagini/es1-scena.png

Passo 3 — Aggiungi il bottone e la scritta

1. Seleziona **Main**, poi in alto nel pannello Scena clicca il **+**, cioè Aggiungi nodo figlio. Cerca **Button**, selezionalo, **Crea**. Rinominalo **BottoneCiao**.
2. Seleziona di nuovo **Main**, **+**, cerca **Label**, **Crea**. Rinominalo **Etichetta**.

Passo 4 — Attacca lo script

Seleziona **Main**. In alto a destra del pannello Scena clicca **Attacca uno script**, l'icona con la pergamena e il **+**. Lascia tutto com'è e premi **Crea**.

Passo 5 — Scrivi il codice

Cancella quello che trovi e incolla:

```
extends Node2D
```

```
@onready var bottone: Button = $BottoneCiao
```

```
@onready var etichetta: Label = $Etichetta
```

```
func _ready() -> void:
```

```
    bottone.position = Vector2(100, 100)
```

```
    bottone.text = "Salutami!"
```

```
    etichetta.position = Vector2(100, 180)
```

```
    etichetta.text = "..."
```

```
    # Colleghiamo il clic, il segnale pressed, alla nostra funzione
```

```
    bottone.pressed.connect(_quando_premo)
```

```
# Questa è come il tuo Button1Click di Lazarus
```

```
func _quando_premo() -> void:
```

```
    etichetta.text = "Ciao! Mi hai premuto."
```

Pezzo per pezzo: `@onready var` prende i due nodi per nome; in `_ready()` diamo posizione e testo; `bottone.pressed.connect(...)` collega il **clic** alla funzione `_quando_premo`, che cambia la scritta.

Passo 6 — Vinci

Premi **F5**. Si apre la finestra: **premi il bottone** e compare “Ciao! Mi hai premuto.” **Ce l’hai fatta.**

SCREENSHOT DA INSERIRE

Il gioco che saluta

immagini/es1-gioca.png

Fallo tuo

- Cambia la frase del saluto con **la tua**.
- Dai un colore alla scritta: dentro `_ready()` aggiungi questa riga, dove i tre numeri sono rosso, verde, blu da 0 a 1: `gdscript etichetta.add_theme_color_override("font_color", Color(1, 0, 0))`

Costruiamo l'Esercizio 2: muovi il quadrato

Qui incontri il concetto più importante di Godot, il **game loop**. Alla fine avrai un **quadrato** che si muove con le frecce.

Passo 1 — Progetto e scena

Come prima: crea un progetto `esercizio2`, poi nel pannello Scena clicca `Scena 2D` e rinomina il nodo radice `Main`.

Passo 2 — Aggiungi il quadrato

Seleziona `Main`, clicca `+`, cerca `ColorRect`, cioè un rettangolo colorato, `Crea`, e rinominalo `Quadrato`.

Passo 3 — Script e codice

Attacca lo script a `Main`, cancella e incolla:

```
extends Node2D

@onready var quadrato: ColorRect = $Quadrato
const VELOCITA: float = 300.0 # pixel al secondo

func _ready() -> void:
    quadrato.size = Vector2(60, 60)
    quadrato.color = Color(0.3, 0.7, 1.0) # azzurro
    quadrato.position = Vector2(200, 200)

# _process gira a OGNI fotogramma: qui muoviamo il quadrato
func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        quadrato.position.x -= VELOCITA * delta
    if Input.is_action_pressed("ui_right"):
        quadrato.position.x += VELOCITA * delta
    if Input.is_action_pressed("ui_up"):
        quadrato.position.y -= VELOCITA * delta
```

```
if Input.is_action_pressed("ui_down"):
    quadrato.position.y += VELOCITA * delta
```

La novità è `func _process(delta):`, che **gira da sola ~60 volte al secondo**. Dentro controlliamo le frecce con `Input.is_action_pressed(...)` e spostiamo il quadrato. `delta` serve a muoversi uguale su ogni computer.

Passo 4 — Vinci

F5, e muovi il quadrato con le **frecce**. *Lazarus reagisce, Godot pulsa.*

SCREENSHOT DA INSERIRE

Il quadrato che si muove
immagini/es2-gioca.png

Fallo tuo

- Cambia il **colore** in `quadrato.color = Color(...)`.
- Cambia la **velocità**: il numero in `const VELOCITA`.

Costruiamo l'Esercizio 3: prendi la moneta

Il primo **mini-gioco vero**: un **cestino** che prende le **monete** che cadono, con **punteggio**. Mette insieme movimento, collisioni e punteggio. È l'idea del vecchio “Chirurgo Pasticcione”, ma più pulita.

Passo 1 — Scena

Crea il progetto `esercizio3`, **Scena 2D**, nodo radice **Main**. Poi aggiungi tre figli a **Main** con il **+**:

- **ColorRect** → rinominalo **Cestino**
- **ColorRect** → rinominalo **Moneta**
- **Label** → rinominalo **Punteggio**

SCREENSHOT DA INSERIRE

La scena con Cestino, Moneta e Punteggio
immagini/es3-scena.png

Passo 2 — Script e codice

Attacca lo script a **Main**, cancella e incolla:

```
extends Node2D

@onready var cestino: ColorRect = $Cestino
@onready var moneta: ColorRect = $Moneta
@onready var punteggio: Label = $Punteggio

const VELOCITA_CESTINO: float = 500.0
const VELOCITA_MONETA: float = 300.0

var punti: int = 0
var larghezza: float

func _ready() -> void:
    larghezza = get_viewport_rect().size.x
    cestino.size = Vector2(120, 24)
```

```

cestino.color = Color(0.6, 0.4, 0.2)
cestino.position = Vector2(larghezza / 2.0 - 60, get_viewport_rect().size.y - 60)
moneta.size = Vector2(30, 30)
moneta.color = Color(1.0, 0.85, 0.1)
_rimetti_in_alto()
punteggio.position = Vector2(20, 20)
_aggiorna_punteggio()

func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        cestino.position.x -= VELOCITA_CESTINO * delta
    if Input.is_action_pressed("ui_right"):
        cestino.position.x += VELOCITA_CESTINO * delta
    cestino.position.x = clamp(cestino.position.x, 0, larghezza - cestino.size.x)
    moneta.position.y += VELOCITA_MONETA * delta
    # Il cestino tocca la moneta?
    if Rect2(cestino.position, cestino.size).intersects(Rect2(moneta.position,
moneta.size)):
        punti += 1
        _aggiorna_punteggio()
        _rimetti_in_alto()
    elif moneta.position.y > get_viewport_rect().size.y:
        _rimetti_in_alto()

func _rimetti_in_alto() -> void:
    var x := randf_range(0, larghezza - moneta.size.x)
    moneta.position = Vector2(x, -moneta.size.y)

func _aggiorna_punteggio() -> void:
    punteggio.text = "Monete: %d" % punti

```

Cosa succede: in `_ready()` prepariamo cestino, moneta e punteggio; in `_process()` muoviamo il cestino con le frecce, facciamo scendere la moneta e con `Rect2(...).intersects(...)` controlliamo se il cestino **tocca** la moneta. Se sì, **+1 punto** e la moneta riparte dall'alto in una colonna a caso.

Passo 3 — Vinci

F5: muovi il cestino con `← →` e **prendi le monete**. Il punteggio sale.

SCREENSHOT DA INSERIRE

Il gioco della moneta
immagini/es3-gioca.png

Fallo tuo

- Cambia i **colori** di cestino e moneta.
- Rendilo più difficile: aumenta `VELOCITA_MONETA`.
- Aggiungi le **vite** e un “Game Over” quando finiscono, come nel vecchio “Chirurgo Pasticcione”.

Il percorso: dagli esercizi al “progetto boss”

Qui non si impara con la teoria astratta, ma **facendo**. Ogni esercizio insegna **un pezzo**; poi arriva un gioco più grande — il “**progetto boss**” — che mette insieme quei pezzi. Ecco la scala che stiamo salendo.

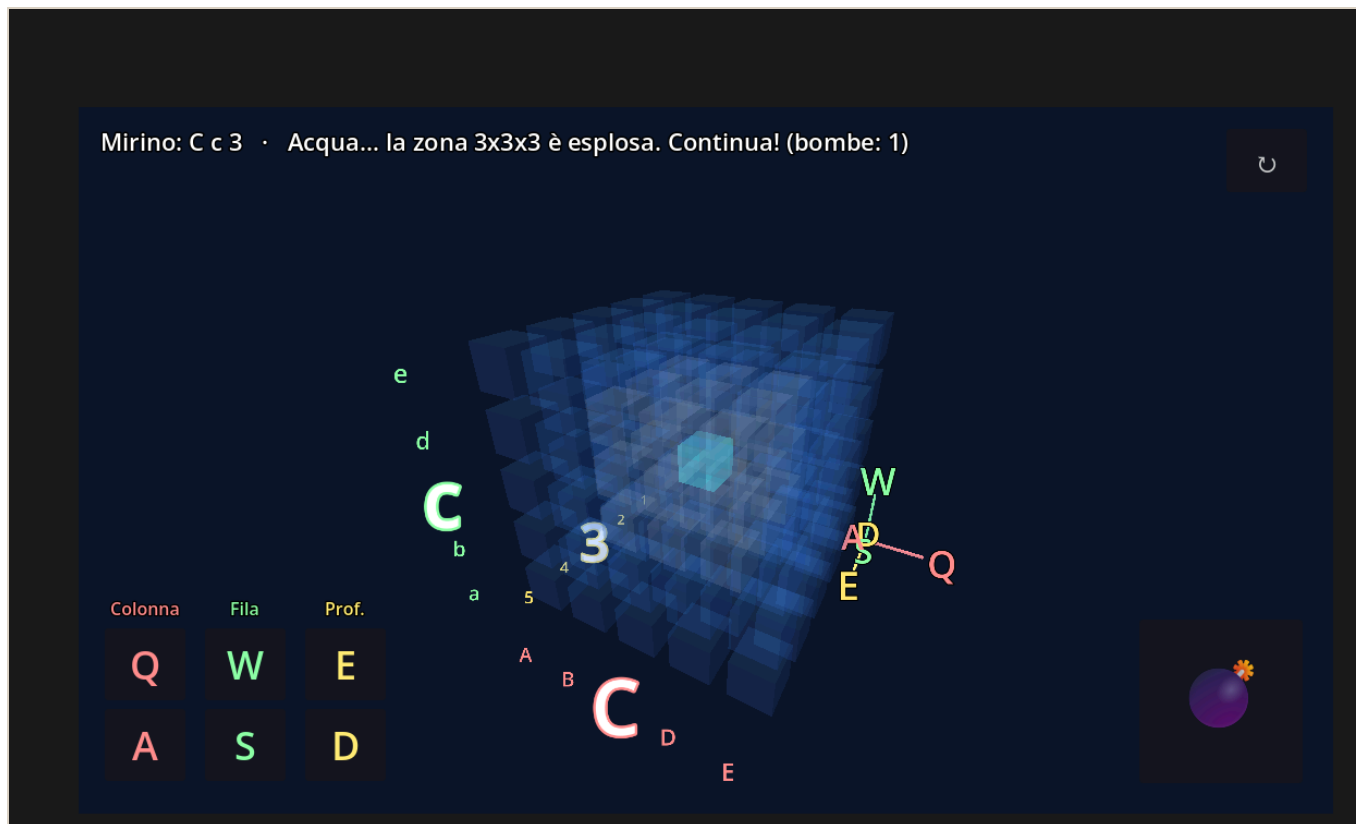
I gradini piccoli: gli esercizi dell’eserciziario

| Esercizio | Cosa impari | Il concetto sotto |
|---------------------------|-------------------------------|---|
| 1 • Il bottone che saluta | Un clic fa succedere qualcosa | Il segnale , il tuo <code>Button1Click</code> di Lazarus |
| 2 • Muovi il quadrato | Far muovere le cose da sole | Il game loop <code>_process(delta)</code> + input |
| 3 • Prendi la moneta | Un mini-gioco vero | Movimento + collisioni + punteggio |

Ognuno è **corto** e finisce con una **vittoria a schermo**: è fatto apposta così, per vincere subito e non mollare.

Cos’è un “progetto boss”

È un gioco **già pronto**, più grosso, che **non si copia riga per riga**. Si **apre, si gioca e si rende proprio**: cambi i colori, il titolo, ci metti una tua foto. È il **premio**: la cosa figa da mostrare subito. Il primo è “**Affonda la Bonomi**”: lo trovi nell’eserciziario e nella cartella `battaglia-navale-3d/`.



Il "progetto boss" Affonda la Bonomi: una battaglia navale in 3D, dentro un cubo d'acqua.

Dal 2D al 3D: cosa cambia nel boss

Finora abbiamo lavorato in **2D**: due coordinate, **x** orizzontale e **y** verticale, un `Vector2`. Il boss è in **3D**: si aggiunge una terza coordinate, **z**, la **profondità**, un `Vector3`. Il campo di gioco non è più una griglia piatta ma un **cubo** di celle.

Due idee nuove, ma **niente panico**:

- **Si costruisce tutto da codice**: le celle, le luci, la telecamera e i bottoni, invece che a mano nell'editor: sono sempre gli stessi **nodi**, solo tanti, creati con un ciclo `for`.
- Sono gli **stessi concetti di prima, in grande**: il **game loop** gira il cubo, l'**input** muove il mirino. Chi ha fatto gli esercizi 2 e 3 ha già visto tutto.

Come affrontarlo, un gradino alla volta

Non devi finire il boss tutto in una volta: **sali un gradino alla volta**, e a ogni gradino ti porti a casa una vittoria vera.

1. **Gioca** e scegli la difficoltà: con 4 è facilissimo.
2. **Cambia un colore** dell'acqua o del mirino: basta cambiare un numero nel codice, e l'effetto è immediato.

3. **Metti la tua foto**, o un meme, al posto di quella di partenza.

4. **Cambia il titolo** del gioco.

5. **Leggi una funzione piccola** e prova a spiegare **a voce** cosa fa, per esempio come si muove il mirino.

6. Se vai forte: cambia la potenza della bomba o aggiungi un secondo sottomarino.

Parti dal gradino che ti riesce: anche solo giocare e cambiare un colore è già una vittoria da mostrare. Vale sempre: **Vinci subito · Fallo tuo · Mostralo.**

Come useremo l'AI

L'AI è come la **calcolatrice in matematica**: aiuta, ma se non capisci cosa stai facendo non serve a niente.

- Usala per: capire un errore, farti spiegare un concetto, avere un suggerimento, uno **spunto da studiare e modificare**.
- Non usarla per: farti scrivere tutto e consegnarlo senza capirlo.
- **Prova del nove**: se sai **spiegare a voce, riga per riga**, il codice che presenti, la competenza c'è.